

Task 2 – Developing the Prototype

Introduction and Development Overview

Purpose of the Prototype:

The whole point of this prototype was to build a working website for Greenfield Local Hub. They are a group of local farmers and food producers. The client wanted a platform where customers could buy and promote local food and see exactly what they were paying for. They also wanted producers to be able to manage their own stock and see what orders had come in.

Client Requirements:

Looking at the task brief, I had to include quite a few things. Firstly the site needed to tell customers about the producers, how they grow their food, and why buying locally is a good thing. Then customers needed to be able to browse products with clear prices and see if something was in stock. They also had to place orders, either for collection or delivery and they needed their own accounts so they could see what they had ordered before.

For the producers, I had to build a dashboard where they could update their stock levels, change product details and view any orders that had been placed for their items.

The client also wanted a few extra features that came from asking their existing customers what they would like. These were order tracking so people could see where their delivery was, a loyalty scheme where you earn points and get discounts, and some accessibility features to help people who might struggle with standard websites. I decided to put in the loyalty scheme properly and added a dark mode and light mode toggle to help with accessibility.

Technologies I Used:

I had to pick some tools to build this prototype and I chose ones that work nicely together and are used a lot in real jobs.

For the frontend – that is the bit the user actually sees and clicks on. I used React with JavaScript. This let me build a site that responds well to different screen sizes, and I could break everything down into small, reusable chunks of code. That also made it easier when I had to fix things.

For the backend, the part that runs on the server, deals with logins, fetches products from the database, and saves orders, I used Node.js with Express. This handled all the requests coming from the frontend and sent back the right data.

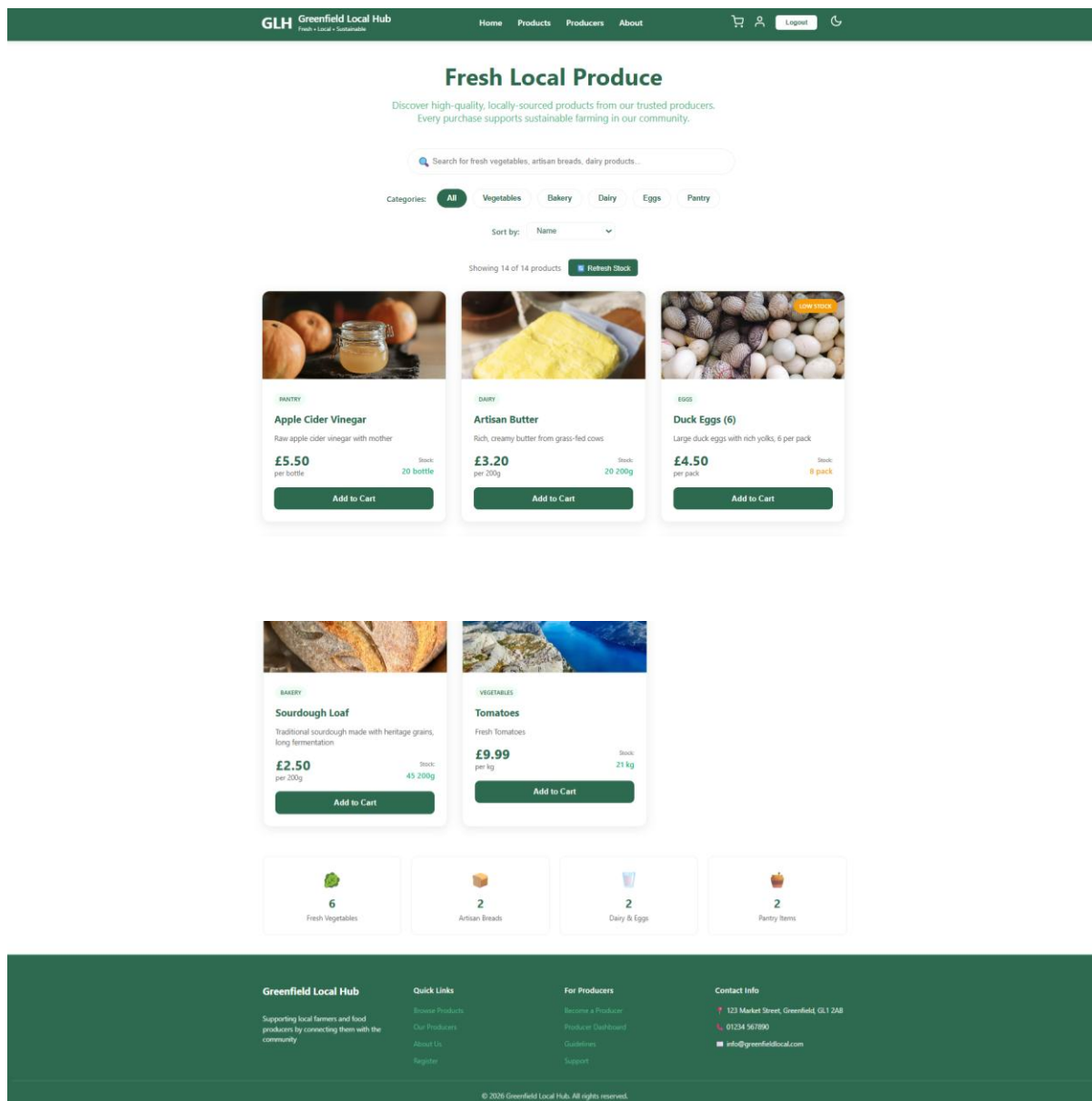
For the database, I went with SQLite3 as it is easy to set up and saves. It saves data to a file on the server, so I did not need to set up a separate database program. That kept things simple but still meant that when I restarted the server, my data did not just disappear.

I also added a map using Leaflet, which works with OpenStreetMap. This map shows where each producer is located for the customers who want to know where their food comes from.

Development Approach:

An iterative approach was used for the development process because there were numerous parts of the website to be developed that should work together including the product page, the shopping basket, and checkout, the customers account information, producer dashboard and loyalty scheme. Doing it all together and testing at the end could be very risky since if anything went wrong, I would not know which component caused the problem.

each phase of the website was done separately until completion after which it was tested. I began by developing the user interface and navigation menu, followed by the products page, the basket and checkout, customer account log-in and sign-up, producer dashboard, then the loyalty scheme and finally the mapping API. As soon as I completed each phase, I conducted a test. Any bug was quickly and immediately addressed and fixed before moving to the next part. This saved me a lot of time since the issues were fixed out before becoming difficult.



```

useEffect(() => {
  // Fetch products from API to get real-time stock data
  setLoading(true);

  axios.get('http://localhost:5001/api/products')
    .then(res => {
      console.log('Products fetched from API:', res.data.length);
      setProducts(res.data);
      setLoading(false);
    })
    .catch(err => {
      console.error('Error fetching products:', err);
    });

```

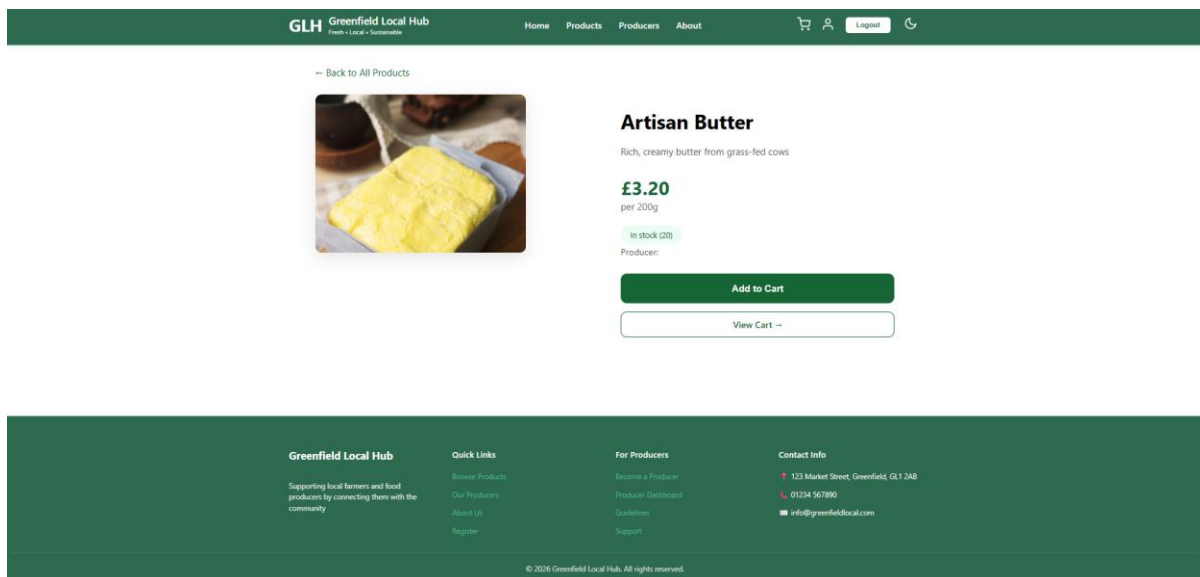
```

const filteredAndSortedProducts = products
  .filter(product => {
    const matchesSearch = product.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
      product.description.toLowerCase().includes(searchTerm.toLowerCase());
    const matchesCategory = selectedCategory === 'All' || product.category === selectedCategory;
    return matchesSearch && matchesCategory;
  })
  .sort((a, b) => {
    switch (sortBy) {
      case 'name':
        return a.name.localeCompare(b.name);
      case 'price-low':
        return a.price - b.price;
      case 'price-high':
        return b.price - a.price;
      case 'stock':
        return b.stock - a.stock;
      default:
        return 0;
    }
  });

```

When the Products page loads it sends a request to my backend to get all the Products from the database. I used axios because it is easy to use. If the request does not work for some reason the page uses a list of Products that I already put in the code. This way the customer can still see some Products of a page that does not work.

Once the Products are loaded the page looks at what the customer types into the search bar and which category button they click. It then shows the customer the products that match what they are looking for. The page also puts the products in order based on what the customer picks from the menu. I decided to do this on the frontend with the products not on the backend. This makes the Products page feel faster because the customer does not have to wait for the server to respond every time they change something. The customer can just keep looking at the Products without any delay.



```
useEffect(() => {
  // Fetch product by ID from API
  const fetchProduct = async () => {
    try {
      const response = await axios.get(`http://localhost:5001/api/products`);
      const products = response.data;
      const foundProduct = products.find(p => p.id === parseInt(id));
      setProduct(foundProduct);
    } catch (error) {
      console.error('Error fetching product:', error);
    }
  };
  fetchProduct();
}, [id]);
```

```
26   if (!product) {
27     return (
28       <div style={{ textAlign: 'center', padding: '100px' }}>
29         <h2>Product not found</h2>
30         <Link to="/products" style={{ color: '#166534' }}>< Back to Products</Link>
31       </div>
32     );
33   }
```

When a customer clicks on a product from the Products page they get taken to the Product Details page. The web address has the product ID number in it like product/5. The code sees this ID. It Then asks my backend for all the products. The code gets the list of products. Then looks through it to find the product with the matching ID. When it finds the product it saves the Product Details so they can be shown on the screen. If a customer types a web address for a product that does not exist or if the product was removed from the database the code will not find the product. In that situation the Product Details page will not show a page or a broken screen. Instead the Product Details page shows a friendly "Product not found" message and

gives the customer a button to go back, to the Products page. This gives the site more professional look.

```
import { useContext } from 'react';
import { CartContext } from '../context/CartContext';
import { useNavigate } from 'react-router-dom';

const Basket = () => {
  const { cart, removeFromCart, updateQuantity, clearCart } = useContext(CartContext);
  const navigate = useNavigate();

  const total = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
  const totalItems = cart.reduce((sum, item) => sum + item.quantity, 0);

  const handleQuantityChange = (id, newQuantity) => {
    if (newQuantity <= 0) {
      removeFromCart(id);
    } else {
      updateQuantity(id, newQuantity);
    }
  };

  const handleCheckout = () => {
    if (cart.length === 0) return;
    navigate('/checkout');
  };

  const handleContinueShopping = () => {
    navigate('/products');
  };
};
```

The Basket page keeps track of everything the customer has added. When the page loads it checks the cart data stored in a shared memory called Context. This Context is where all pages can get information so items stay in the basket even when the customer moves to another page. The total price is calculated by adding up the price multiplied by the quantity for each item in the basket. This is done using a function that loops through every item. The total number of items is calculated by adding up the quantities. When the customer changes the quantity of an item using the plus and, minus buttons the code checks the quantity. If the customer tries to set the quantity below one the item gets removed from the basket. Otherwise the quantity just gets updated. I did this to avoid confusing the customer with a quantity of zero or less. The page also shows how many loyalty points the customer will earn from this order. The customer earns ten points for every pound spent. For example if the total is £17.40 the customer earns 174 points. This updates instantly as the customer changes quantities so they can always see what they will get.

GLH Greenfield Local Hub

Fresh • Local • Sustainable

[Home](#)
[Products](#)
[Producers](#)
[About](#)

[Login](#)

Checkout

Complete your order for fresh local produce

Delivery Method

☒ **Home Delivery**
Get your order delivered to your doorstep (+£3.50)

☐ **Click & Collect**
Collect your order from our hub (free)

Delivery Address

Street Address *

Enter your full address

Loyalty Points Redemption

You have 0 loyalty points available

Select a reward to redeem:

Order Notes (Optional)

Any special instructions for your order...

Place Order • £20.90

Order Summary

Artisan Butter	£6.40
2 x £3.20	
Apple Cider Vinegar	£11.00
2 x £5.50	
Subtotal	£17.40
Delivery	£3.50
Total	£20.90

Earn 200 loyalty points
10 points per £1 spent

[← Back to Basket](#)

Greenfield Local Hub

Supporting local farmers and food producers by connecting them with the community

Quick Links

[Browse Products](#)
[Our Producers](#)
[About Us](#)
[Register](#)

For Producers

[Become a Producer](#)
[Producer Dashboard](#)
[Guidelines](#)
[Support](#)

Contact Info

123 Market Street, Greenfield, GL1 2AB

01234 567890

info@greenfieldlocal.com

```
// Calculate discounts based on selected reward
let discount = 0;
let finalDeliveryFee = deliveryFee;

if (selectedReward) {
  if (selectedReward.type === 'discount') {
    discount = selectedReward.value; // £5 off
  } else if (selectedReward.type === 'freeDelivery') {
    finalDeliveryFee = 0; // Free delivery
  } else if (selectedReward.type === 'percentage') {
    discount = subtotal * 0.15; // 15% off
  }
}

const total = subtotal + finalDeliveryFee - discount;

const currentPoints = currentUser?.points || user?.points || 0;
```

```

try {
  // Get user ID from authentication
  if (!user || !user.id) {
    throw new Error('User not authenticated');
  }

  const orderData = {
    user_id: user.id,
    items: cart.map(item =>({
      product_id: item.id,
      quantity: item.quantity,
      price: item.price
    })),
    total: total,
    order_type: orderType,
    delivery_address: orderType === 'delivery' ? formData.deliveryAddress : null,
    collection_time: orderType === 'collection' ? formData.collectionTime : null,
    pointsUsed: selectedReward ? selectedReward.points : 0
  };

  const response = await axios.post('http://localhost:5001/api/orders', orderData);

  if (response.data.success) {
    clearCart();
    navigate('/account', {
      state: {
        orderSuccess: true,
        orderId: response.data.orderId,

```

The Checkout page is where the customer finishes their order on the Checkout page. The first thing the Checkout page does is ask the customer whether they want home delivery or click and collect on the Checkout page. If the customer picks delivery on the Checkout page they have to fill in their address and pay a £3.50 for the delivery on the Checkout page. If the customer picks collection on the Checkout page they choose a time slot from the list. There is no delivery fee for the collection on the Checkout page. The Checkout page also lets the customer use their loyalty points on the Checkout page. If the customer has loyalty points on the Checkout page they can pick from three rewards on the Checkout page: £5 off free delivery or 15% off the whole order on the Checkout page. The code checks which reward the customer picked on the Checkout page and works out the discount for the customer on the Checkout page. For a percentage discount on the Checkout page it multiplies the subtotal by 0.15 for the customer on the Checkout page. For delivery on the Checkout page it sets the delivery fee to zero for the customer on the Checkout page. The final total is worked out by adding the subtotal and the delivery fee for the customer on the Checkout page then taking away the discount for the customer on the Checkout page. When the customer clicks the place order button on the Checkout page the Checkout page collects all the information for the customer. This includes the customers ID, the list of items in their basket the amount, whether they chose delivery or collection on the Checkout page and how many loyalty points they used on the Checkout page. It then sends all this to my backend using a POST request, from the Checkout page. If everything goes well the backend saves the order to the database clears the basket and sends the customer to their account page so they can see their order history on the account page.

GLH

Greenfield Local Hub

[Home](#)
[Products](#)
[Producers](#)
[About](#)

[Login / Register](#)

Login to Your Account

Email Address

rah@gmail.com

Password

Login

Don't have an account? [Register here](#)

Greenfield Local Hub

Supporting local farmers and food producers by connecting them with the community

Quick Links

[Browse Products](#)

[Our Producers](#)

[About Us](#)

[Register](#)

For Producers

[Become a Producer](#)

[Producer Dashboard](#)

[Guidelines](#)

[Support](#)

Contact Info

123 Market Street, Greenfield, GL1 2AB

01234 567890

info@greenfieldlocal.com

© 2020 Greenfield Local Hub. All rights reserved.

```

const Login = () => {
  const handleSubmit = async (e) => {

    // Validate all fields
    const validationErrors = getValidationErrors(formData, 'login');
    if (Object.keys(validationErrors).length > 0) {
      setErrors(validationErrors);
      return;
    }

    setLoading(true);

    try {
      const response = await axios.post('http://localhost:5001/api/login', formData);

      if (response.data.token) {
        console.log('=== LOGIN DEBUGGING ===');
        console.log('Login response:', response.data);
        console.log('User data from server:', response.data.user);
        console.log('=== END LOGIN DEBUGGING ===');

        login(response.data.token, response.data.user);

        // Role-based redirect
        const userRole = response.data.user.role;

        if (userRole === 'producer') {
          navigate('/dashboard');
        } else {
          navigate('/account'); // or '/' for home
        }
      }
    } catch (error) {
      console.log('Login failed:', error);
      setErrors({ login: 'Invalid email or password' });
    }
  }
}

```

The Login page lets customers and producers sign into their accounts on the Login page. When the user fills in their email and password and clicks the Login button on the Login page the Login page first checks that both fields have been filled in by the user on the Login page. It does this by calling a validation function to validate the fields on the Login page. If something is missing on the Login page it shows an error message on the screen. Stops the form from

sending anything to the server from the Login page. This saves the user from waiting for a server response when they have clearly missed a field on the Login page. If the validation passes on the Login page the login page sends a POST request to my backend with the email and password from the login page. The backend then checks if there is a user with that email in the database and verifies the password for the user on the login page. If the details are correct for the user on the login page the backend sends back a token and the users information to the Login page. The frontend saves this user data using a login function from something called an AuthContext on the Login page. This Context keeps the user logged in as they move between pages on the website. After a login on the login page the page looks at the users role on the Login page. If the role is "producer" for the user on the login page it sends them to the Dashboard where they can manage their products and see orders for the producer. If the role is "customer" for the user on the login page it sends them to the Account page where they can see their order history and loyalty points for the customer. This way each type of user sees the pages that're relevant to them on the website.

GLH

Greenfield Local Hub

Fresh • Local • Sustainable

[Home](#) [Products](#) [Producers](#) [About](#)

Login / Register

Create Your Account

I want to:

Customer - I want to buy local products

Full Name *

Enter your full name

Email *

Enter your email

Password (min 8 characters) *

Create a password

Confirm Password *

Confirm your password

Create Account

Already have an account? [Login here](#)

Greenfield Local Hub

[Quick Links](#)

[For Producers](#)

[Contact Info](#)

```

// Basic validations
if (!formData.full_name.trim()) {
  setError("Full name is required");
  return;
}

if (!formData.email.trim()) {
  setError("Email is required");
  return;
}

// Email format validation
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(formData.email)) {
  setError("Please enter a valid email address");
  return;
}

if (formData.password !== formData.confirmPassword) {
  setError("Passwords do not match");
  return;
}

if (formData.password.length < 8) {
  setError("Password must be at least 8 characters");
  return;
}

```

```

if (response.data.success) {
  // Auto-login after successful registration
  try {
    const loginResponse = await axios.post('http://localhost:5001/api/login', {
      email: formData.email,
      password: formData.password
    });

    if (loginResponse.data.token) {
      // Store token and user info
      localStorage.setItem('token', loginResponse.data.token);
      localStorage.setItem('user', JSON.stringify(loginResponse.data.user));

      alert(`Account created successfully as ${formData.role}! You are now logged in.`);

      // Redirect based on role
      if (formData.role === 'producer') {
        navigate('/dashboard'); // Producer goes to dashboard
      } else {
        navigate('/account'); // Customer goes to account page
      }
    }
  } catch (loginErr) {
    // If auto-login fails, still show success and redirect to login
    alert(`Account created successfully as ${formData.role}! Please login to continue.`);
    navigate('/login');
  }
}

```



```

useEffect(() => {
  if (!user || !user.id) {
    setLoading(false);
    setOrders([]);
    setCurrentUser(null);
    return;
  }

  // Fetch fresh user data and orders
  Promise.all([
    axios.get(`http://localhost:5001/api/users/${user.id}`),
    axios.get(`http://localhost:5001/api/orders/${user.id}`)
  ])
    .then(([userRes, ordersRes]) => {
      setCurrentUser(userRes.data);
      setOrders(ordersRes.data);
      setLoading(false);
    })
    .catch(err => {
      console.error('Error fetching account data:', err);
      setLoading(false);
    });
}, [user?.id]);

```

```

if (loading) {
  return (
    <div style={{ maxWidth: '1000px', margin: '40px auto', padding: '0 30px' }}>
      <div style={{ textAlign: 'center', padding: '60px 20px' }}>
        <p style={{ color: '■ #666', fontSize: '18px' }}>Loading account information...</p>
      </div>
    </div>
  );
}

return (
  <div style={{ maxWidth: '1000px', margin: '40px auto', padding: '0 30px' }}>
    <div style={{ marginBottom: '40px' }}>
      <h1 style={{ fontSize: '42px', color: '■ #2d6a4f', marginBottom: '10px', fontWeight: '700' }}>My Account</h1>
      <p style={{ color: '■ #52b788', fontSize: '18px' }}>
        Welcome back, {user?.full_name || 'Customer'}!
      </p>
    </div>

    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '30px', marginBottom: '40px' }}>
      <div style={{
        background: 'linear-gradient(135deg, ■ #2d6a4f 0%, ■ #52b788 100%)',
        padding: '30px',
        borderRadius: '16px',
        color: 'white',
        boxShadow: '0 4px 20px ■ rgba(45, 106, 79, 0.3)'
      }}>
        <h3 style={{ fontSize: '24px', marginBottom: '15px', fontWeight: '600' }}>Loyalty Points</h3>
      </div>
    </div>
  </div>
);

```

```

<div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start', marginBottom: '20px' }}>
  <div>
    <h4 style={{ color: '#2d6a4f', fontSize: '20px', marginBottom: '8px' }}>
      Order #{order.id}
    </h4>
    <p style={{ color: '#666', margin: '0', fontSize: '14px' }}>
      {formatDate(order.created_at)}
    </p>
    <p style={{ color: '#666', margin: '5px 0 0', fontSize: '14px' }}>
      {order.order_type === 'delivery' ? '🚚 Delivery' : '📦 Collection'}
      {order.order_type === 'delivery' && order.delivery_address && ` to ${order.delivery_address}`}
      {order.order_type === 'collection' && order.collection_time && ` at ${order.collection_time}`}
    </p>
  </div>
  <div style={{ textAlign: 'right' }}>
    <div style={{ fontSize: '24px', fontWeight: '700', color: '#2d6a4f', marginBottom: '8px' }}>
      £{order.total.toFixed(2)}
    </div>
    <span style={{

```

The Account page is where customers can see their order history and loyalty points. When the page loads it checks if a user is logged in. If there is a logged in user it sends two requests to the backend at the time. This means the Account page gets the users information, including their loyalty points balance and a list of all the orders they have placed in the past all at the same time. The Account page gets this information using something called promise.all. This is faster than waiting for one request to finish before starting the other. The Account page shows the loyalty points in a green box at the top of the page. The number of loyalty points comes from the users record in the database.

Below that the Account page has a section showing rewards. The customer can see that 100 loyalty points gets them £5 off 250 loyalty points gets delivery and 500 loyalty points gets 15 per cent off their whole order. The Account page has a button but it is disabled if the customer does not have enough loyalty points. The button text tells them how more loyalty points they need.

Below the rewards section the Account page shows every order the customer has placed. Each order card shows the order number, the date it was placed whether it was delivery or collection the amount and the current status. The status has a colour code so the customer can quickly see if their order is still pending, being prepared, ready for collection or already delivered.

Producer Dashboard

Manage your products, view orders and track your sales

My Products

View Orders

Product Management

Add New Product

Product	Category	Price	Stock	Status
Potatoes kg	Vegetables	£ 10	20	In Stock

Producer Dashboard

Manage your products, view orders and track your sales

My Products

View Orders

Customer Orders

Order #3 reghan • 2026-04-26 18:01:14 Pending	£10.00 pending
--	--------------------------------------

```

useEffect(() => {
  console.log('=== DASHBOARD DEBUGGING ===');
  console.log('User data from context:', user);
  console.log('User ID:', user?.id);
  console.log('User role:', user?.role);
  console.log('User producer_id:', user?.producer_id);
  console.log('User business_name:', user?.business_name);

  // Check localStorage
  const token = localStorage.getItem('token');
  const userStorage = localStorage.getItem('user');
  console.log('Token from localStorage:', token);
  console.log('User from localStorage:', userStorage ? JSON.parse(userStorage) : null);

  if (user && user.role === 'producer' && user.producer_id) {
    console.log('Producer found! Using producer_id:', user.producer_id);

    // Create producer info from user data (no API call needed)
    const producerData = {
      id: user.producer_id,
      user_id: user.id,
      farm_name: user.business_name || user.full_name,
      location: user.address || '',
      methods_description: user.business_desc || 'Local producer',
      logo_url: 'https://picsum.photos/id/999/100/100',
      full_name: user.full_name,
      business_name: user.business_name,
      business_desc: user.business_desc
    };
  }
};

```

```

const handleAddProduct = () => {
  console.log('Adding product:', newProduct);
  console.log('Producer info:', producerInfo);

  // Check required fields
  if (!newProduct.name || !newProduct.price || !newProduct.stock) {
    alert("Please fill Product Name, Price and Stock");
    return;
  }

  // Check if producer info is available
  if (!producerInfo) {
    alert("Producer information not loaded. Please wait a moment and try again.");
    return;
  }

  // Validate numeric values
  if (isNaN(parseFloat(newProduct.price)) || parseFloat(newProduct.price) <= 0) {
    alert("Please enter a valid price greater than 0");
    return;
  }

  if (isNaN(parseInt(newProduct.stock)) || parseInt(newProduct.stock) < 0) {
    alert("Please enter a valid stock quantity (0 or greater)");
    return;
  }
}

```

```

const updateStock = (id, newStock) => {
  const product = myProducts.find(p => p.id === id);
  if (product) {
    axios.put(`http://localhost:5001/api/products/${id}`, {
      ...product,
      stock: parseInt(newStock) || 0
    }).then(() => {
      // Update local state
      setMyProducts(myProducts.map(p =>
        p.id === id ? { ...p, stock: parseInt(newStock) || 0 } : p
      ));
    }).catch(err => console.error('Error updating stock:', err));
  }
};

const updatePrice = (id, newPrice) => {
  const product = myProducts.find(p => p.id === id);
  if (product) {
    axios.put(`http://localhost:5001/api/products/${id}`, {
      ...product,
      price: parseFloat(newPrice) || 0
    }).then(() => {
      // Update local state
      setMyProducts(myProducts.map(p =>
        p.id === id ? { ...p, price: parseFloat(newPrice) || 0 } : p
      ));
    }).catch(err => console.error('Error updating price:', err));
  }
};

```

The Producer Dashboard is the tool for farmers and food makers to manage their products. When a producer logs in it checks what role they have and gets their producer ID. It then uses this ID to get the products that belong to that producer from the database. So a new producer sees a list while an existing producer sees only their own products. This is because producers should only manage their stock. The dashboard has two tabs. The first tab shows all the producers products in a table. Next to each product there are boxes where the producer can change the price or stock level. When they type a number and move away it updates the backend and the change saves right away. This makes it quick and easy to adjust prices or mark items as stock. There is also an "Add New Product" button that opens a form. When the producer fills in the details and submits it sends the product to the backend with their producer ID. The new product then appears in their list immediately. The second tab is for viewing orders. This shows any customer orders that include the producers products. The producer can see the order number, customer name, total amount and current status. There is a menu to each order so the producer can update the status. They can change it from "pending" to "preparing" to "ready" for collection. This gives customers updated information about when their stuff will be ready.

About Greenfield Local Hub

We are a cooperative of local farmers and food producers committed to bringing fresh, sustainable, and traceable food to our community.



Our Mission

To strengthen connections between local food producers and our community

Why Buy Local?

Discover the benefits of supporting your local food system



Support Local Economy

Money stays in the community, creating jobs and supporting local families. Every purchase strengthens our local economy.



Environmental Benefits

Reduced food miles and lower carbon emissions. Local food means less transportation and fresher, more sustainable choices.



Freshness & Quality

Picked at peak ripeness with more nutrients and better flavor. Local produce is fresher and tastes better.

How We Work

Our simple process connects producers directly with customers

1



Producers Join

Register and verify credentials

2



Customers Browse

Discover and order products

3



We Coordinate

Manage orders and stock levels

4



Everyone Benefits

Fresh food, strong local economy

Our Commitments

The values that guide everything we do



Full Transparency

Complete traceability from farm to table



Fair Prices

Fair compensation for producers, good value for customers



Quality Standards

Rigorous quality checks and sustainable practices



Accessibility

Fresh, local food accessible to everyone

Our Impact

Making a real difference in our community

25+

Local Producers

2000+

Active Customers

50,000+

Orders Fulfilled

£250k

Revenue to Local Producers

Our Producer Locations

Find our local producers and see where your fresh food comes from



Greenfield Local Hub

Supporting local farmers and food producers by connecting them with the community

Quick Links

[Browse Products](#)
[Our Producers](#)
[About Us](#)
[Register](#)

For Producers

[Become a Producer](#)
[Producer Dashboard](#)
[Guidelines](#)
[Support](#)

Contact Info

[123 Market Street, Greenfield, GL1 2AB](#)
[01234 567890](#)
info@greenfieldlocal.com

© 2026 Greenfield Local Hub. All rights reserved.

```

const About = () => {
  const mapRef = useRef(null);
  const { darkMode } = useContext(ThemeContext);

  useEffect(() => {
    // Initialize Leaflet map with delay to ensure DOM is ready
    const initializeMap = () => {
      if (typeof window !== 'undefined' && !mapRef.current && document.getElementById('map')) {
        try {
          // Create map centered on Greenfield area
          const map = L.map('map').setView([52.105, -2.295], 13);

          // Add OpenStreetMap tiles
          L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
            attribution: 'OpenStreetMap contributors'
          }).addTo(map);

          // Define producer locations with custom icons
          const producers = [
            {
              name: "Thompson's Organic Farm",
              location: "Greenfield Valley",
              lat: 52.1,
              lng: -2.3,
              products: "Organic vegetables, salad leaves, honey"
            },
            // Create custom green icon for markers
            const greenIcon = L.divIcon({
              html: '<div style="background-color: #2d6a4f; color: white; width: 30px; height: 30px; border-radius: 50%; display: flex; align-items: center; justify-content: center; font-size: 1em; font-weight: bold;">🌿</div>',
              iconSize: [30, 30],
              className: 'custom-marker'
            });

            // Add markers for each producer
            producers.forEach(producer => {
              const marker = L.marker([producer.lat, producer.lng], { icon: greenIcon })
                .addTo(map)
                .bindPopup(`
                <div style="font-family: Arial, sans-serif;">
                <h4 style="margin: 0 0 8px 0; color: #2d6a4f; font-size: 16px;">${producer.name}</h4>
                <p style="margin: 0 0 8px 0; color: #666; font-size: 14px;">📍 ${producer.location}</p>
                <p style="margin: 0; color: #52b788; font-size: 13px; font-weight: 600;">🥬 ${producer.products}</p>
                </div>
              `);

              // Open popup on hover
              marker.on('mouseover', function() {
                this.openPopup();
              });
            });
          }
        } catch (error) {
          console.error('Error initializing map:', error);
        }
      }
    };

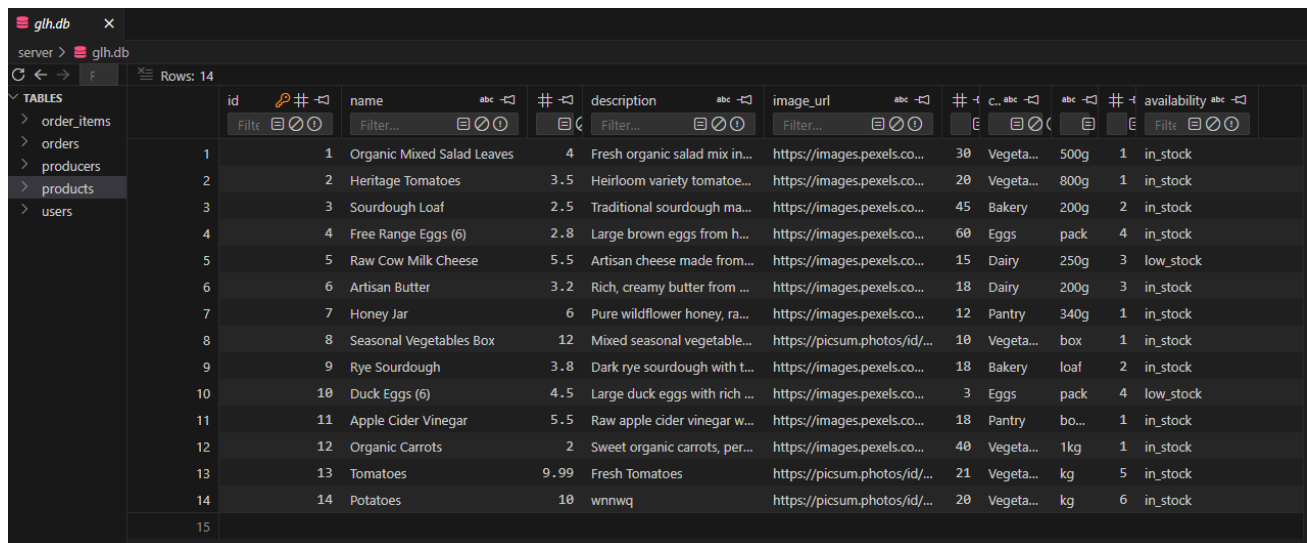
    initializeMap();
  });
}

```

I chose to add an interactive map using Leaflet, which works with free map data from OpenStreetMap. When the page loads, the map initialises and centres itself on the Greenfield area using the coordinates 52.105 latitude and -2.295 longitude. This shows customers roughly where the local farms are located.

I created a list of four producers, each with their name, location, what they produce, and their latitude and longitude coordinates. For each producer, the code adds a marker to the map. I made a custom green marker with a little leaf emoji so it fits with the local food theme. When a customer clicks on a marker, a card appears showing the producers name and basically where they are based and what products they sell. I also added a hover effect so the card opens when the mouse moves over the marker which makes it feel more interactive. The map is completely free to use and does not need an API key and works straight away. Customers can zoom in and

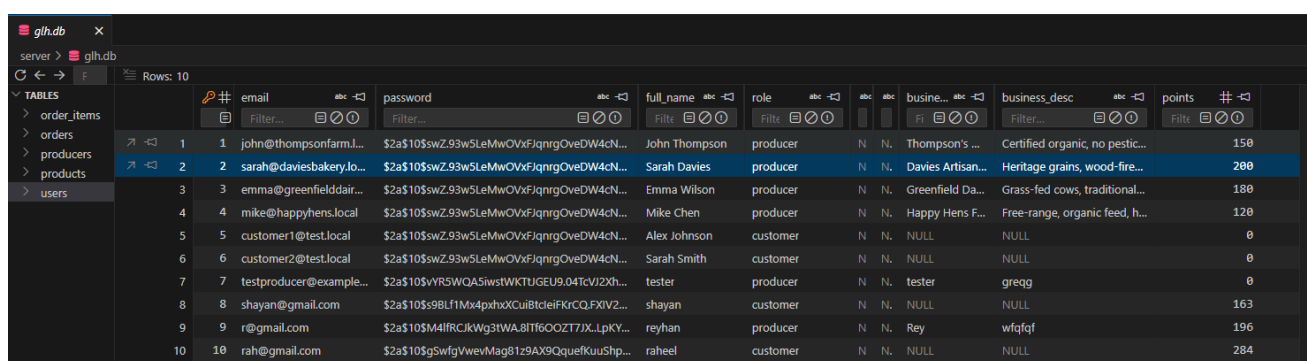
out, drag the map around, and click on markers to learn more about where their food comes from.



The screenshot shows a database viewer interface for 'glh.db'. The 'products' table is selected, displaying 14 rows of data. The table has columns: id, name, price, description, image_url, category, weight, and availability. The data includes various food items like Organic Mixed Salad Leaves, Heritage Tomatoes, Sourdough Loaf, etc.

id	name	price	description	image_url	category	weight	availability
1	Organic Mixed Salad Leaves	4	Fresh organic salad mix in...	https://images.pexels.co...	Vegeta...	500g	in_stock
2	Heritage Tomatoes	3.5	Heirloom variety tomatoe...	https://images.pexels.co...	Vegeta...	800g	in_stock
3	Sourdough Loaf	2.5	Traditional sourdough ma...	https://images.pexels.co...	Bakery	200g	in_stock
4	Free Range Eggs (6)	2.8	Large brown eggs from h...	https://images.pexels.co...	Eggs	pack	in_stock
5	Raw Cow Milk Cheese	5.5	Artisan cheese made from...	https://images.pexels.co...	Dairy	250g	low_stock
6	Artisan Butter	3.2	Rich, creamy butter from ...	https://images.pexels.co...	Dairy	200g	in_stock
7	Honey Jar	6	Pure wildflower honey, ra...	https://images.pexels.co...	Pantry	340g	in_stock
8	Seasonal Vegetables Box	12	Mixed seasonal vegetable...	https://picsum.photos/id/...	Vegeta...	box	in_stock
9	Rye Sourdough	3.8	Dark rye sourdough with t...	https://images.pexels.co...	Bakery	loaf	in_stock
10	Duck Eggs (6)	4.5	Large duck eggs with rich ...	https://images.pexels.co...	Eggs	pack	low_stock
11	Apple Cider Vinegar	5.5	Raw apple cider vinegar w...	https://images.pexels.co...	Pantry	bo...	in_stock
12	Organic Carrots	2	Sweet organic carrots, per...	https://images.pexels.co...	Vegeta...	1kg	in_stock
13	Tomatoes	9.99	Fresh Tomatoes	https://picsum.photos/id/...	Vegeta...	kg	in_stock
14	Potatoes	10	winnq	https://picsum.photos/id/...	Vegeta...	kg	in_stock

Products table holds information about all the items present on the website. As seen from the screenshot above. There is a unique id number, name, price, description, and image URL assigned to each product. There are 14 products in total at the moment, such as Organic Mixed Salad Leaves, Heritage Tomatoes, Sourdough Loaf etc. Before placing the order the software will check the availability of the item in the inventory. Incasse if there is not enough stock of the required product, the software will reject the order. the website will not sell out of stock items. The image URLs point to the free images found on Pexels.com. To make the database lighter and to increase website loading speed, I decided to save only URLs in the database and leave the actual images on Pexels website.

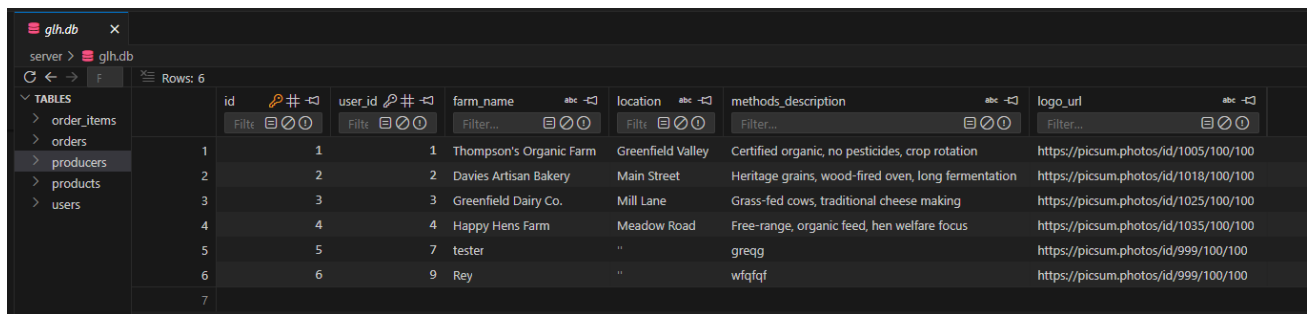


The screenshot shows a database viewer interface for 'glh.db'. The 'users' table is selected, displaying 10 rows of data. The table has columns: id, email, password, full_name, role, business_name, business_description, and points. The data includes users like John Thompson, Sarah Davies, Emma Wilson, etc.

id	email	password	full_name	role	business_name	business_description	points
1	john@thompsonfarm.l...	\$2a\$10\$swZ.93w5LeMwOVxqjqrngOveDW4cN...	John Thompson	producer	N. N.	Thompson's ...	150
2	sarah@daviesbakery.lo...	\$2a\$10\$swZ.93w5LeMwOVxqjqrngOveDW4cN...	Sarah Davies	producer	N. N.	Davies Artisan...	200
3	emma@greenfielddair...	\$2a\$10\$swZ.93w5LeMwOVxqjqrngOveDW4cN...	Emma Wilson	producer	N. N.	Greenfield Da...	180
4	mike@happyhens.local	\$2a\$10\$swZ.93w5LeMwOVxqjqrngOveDW4cN...	Mike Chen	producer	N. N.	Happy Hens F...	120
5	customer1@test.local	\$2a\$10\$swZ.93w5LeMwOVxqjqrngOveDW4cN...	Alex Johnson	customer	N. N.	NULL	0
6	customer2@test.local	\$2a\$10\$swZ.93w5LeMwOVxqjqrngOveDW4cN...	Sarah Smith	customer	N. N.	NULL	0
7	testproducer@example...	\$2a\$10\$VYR5WQA5iwtWKTU9.04TcVj2Xh...	tester	producer	N. N.	tester	0
8	shayan@gmail.com	\$2a\$10\$98L1TmX4pXhXCUi8tcleIFKRCQ.FXIV2...	shayan	customer	N. N.	NULL	163
9	r@gmail.com	\$2a\$10\$M4lFRcJkKg3tWA.8lTf6OOZT7JX..LpKY...	reyhan	producer	N. N.	Rey	196
10	rah@gmail.com	\$2a\$10\$gSwfgVvevMag8tZ9AX9QquefKuuShp...	raheel	customer	N. N.	NULL	284

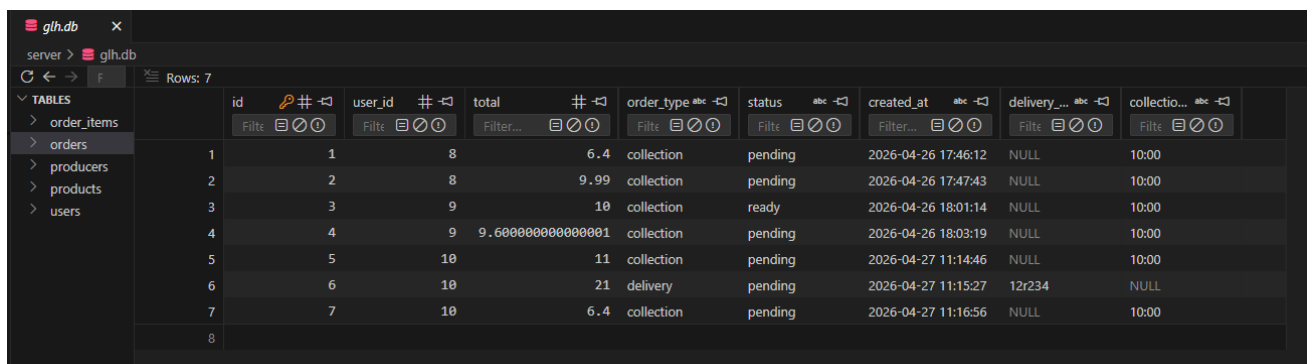
The users table keeps record of all accounts created by the users in the application. From the screenshot you can see, each user is allocated a unique ID, an email, hash value for the password, name, role type and points. The passwords are saved as hash values using bcrypt. So even if there was any leakage of the information kept in the database no one could retrieve the real passwords due to the encryption applied. It meets GDPR standards. The role value will either be 'customer' or 'producer.' This decides which pages the users will have access to. Like

customers have access to shopping and account history page while producers are directed to the dashboard where they manage their product list and orders.



	id	user_id	farm_name	location	methods_description	logo_url
1	1	1	Thompson's Organic Farm	Greenfield Valley	Certified organic, no pesticides, crop rotation	https://picsum.photos/id/1005/100/100
2	2	2	Davies Artisan Bakery	Main Street	Heritage grains, wood-fired oven, long fermentation	https://picsum.photos/id/1018/100/100
3	3	3	Greenfield Dairy Co.	Mill Lane	Grass-fed cows, traditional cheese making	https://picsum.photos/id/1025/100/100
4	4	4	Happy Hens Farm	Meadow Road	Free-range, organic feed, hen welfare focus	https://picsum.photos/id/1035/100/100
5	5	7	tester	"	gregg	https://picsum.photos/id/999/100/100
6	6	9	Rey	"	wfqfqf	https://picsum.photos/id/999/100/100
7						

The producers table holds data for each individual producer like the farmer or the manufacturer that sells produce using the website. The screenshot shows the details that have been saved for each producer. these include an ID number, user ID, business name, location, and logo image URL. The user ID provides a relationship between the producers login information in the users table and the details of his or her business. This ensures that each product belongs to one producer and the system can match producers with their products when logging into their account. there are four producers in the site which include Thompson's Organic Farm, Davies Artisan Bakery, Greenfield Dairy Co. and Happy Hens Farm. All four producers have their own locations within Greenfield as shown in the map provided in the About page of the site.



	id	user_id	total	order_type	status	created_at	delivery_address	collection_time
1	1	8	6.4	collection	pending	2026-04-26 17:46:12	NULL	10:00
2	2	8	9.99	collection	pending	2026-04-26 17:47:43	NULL	10:00
3	3	9	10	collection	ready	2026-04-26 18:01:14	NULL	10:00
4	4	9	9.600000000000001	collection	pending	2026-04-26 18:03:19	NULL	10:00
5	5	10	11	collection	pending	2026-04-27 11:14:46	NULL	10:00
6	6	10	21	delivery	pending	2026-04-27 11:15:27	12r234	NULL
7	7	10	6.4	collection	pending	2026-04-27 11:16:56	NULL	10:00
8								

Orders table has all the orders made by the customers. An order contains order ID, user ID referring to the users table, order date, total amount, order status, and order type. Order status field has one of the values like pending, preparing, ready, delivered, and completed. This way customers can see their order status which was one of the requirements identified from the clients market research. The order type field has either delivery or collection based on the choice of the customer. If it is a delivery then the order will have a delivery address and while for the collection option an order will have a pickup time.

	id	order...	produ...	quantity	price_at_...
1	1	1	6	2	3.2
2	2	2	13	1	9.99
3	3	3	14	1	10
4	4	4	6	3	3.2
5	5	5	11	2	5.5
6	6	6	10	5	4.5
7	7	7	6	2	3.2
8					

Order Items table contains details about the items that have been used in the order. ID column indicates a unique ID. order_id refers to the ID column in Orders table while product_id refers to the ID column in products. Other columns include quantity of the ordered item and the price at which the item has been ordered. The last column becomes important since prices for the products may change in the future. So with this information the organization can be certain about the price that has been used in the ordering process even if the price is raised to £4.00 from £3.20 per item.

Maintainable and Readable Code:

To keep things easy, I have added comments explaining the role of key functions within the code. For example in server.js of the backend, the comment at the beginning of an endpoint for creating orders describes how the price is calculated, stock checked, order saved, inventory decreased, and loyalty points rewarded. the second developer would be able to get an idea about what exactly is going on while scanning the code.

This project follows certain naming conventions like when working on JavaScript all variable names are written according to the camelCase convention, e.g., fetchProducts, addToCart, and userEmail. Also when using SQLite, tables and column names are written in snake_case, e.g., order_items, producer_id, and full_name as it makes the code look more professional and easy to follow.

With modules, changes to the site can be made easily. There is an individual file for each web page which includes Products.js, Basket.js, and Dashboard.js etc. Then there are the navigation bar and footer modules which I have used on every page. If at any point in the future the footer required changes I will not have to make them in multiple files because all that needs to be done is change one file. It not only saves time but it also reduces the possibility of errors. The code is scalable as well which means the client in the future requires additional functionalities such as product reviews or chats, another developer will be able to pick up from my code and continue easily.

Accessibility and User Experience:

I wanted the website to feel easy and natural to use and not confusing or frustrating. From the very start I putted myself and thought about how a real customer or producer would actually use the site. Would they know where to click? Would they understand what each page does? Would they get annoyed by anything? I tried to answer all of these questions while building the prototype.

The layout of the site stays the same on every single page. The navigation bar is always at the top with the logo on the left and the login/register button on the right. The footer is always at the bottom with quick links and contact information. This might sound simple but it is important because customers do not have to relearn where things are every time they go to a different page. They know that "Products" will always be in the same spot, and "Account" will always be where they expect it to be.

The choice to use earth and natural colours was because the fact that the Greenfield Local Hub (GLH) focuses mainly on selling fresh produce and vegetables so the website should evoke feelings similar to those when people visit a local store and not a supermarket chain. I chose green colour to dominate as it represents a fresh atmosphere. The dark forest green is used for buttons, headings and interactive areas such as the 'Add to Cart' button and 'Login' button etc, however it does not interfere with the overall visual designs of the website because of its soft appearance. As for the background colour it is white although very light and was chosen because it gives a pleasant vibe and makes the web pages look clean and fresh. The dark grey is selected for the text color since the black one may hurt the eyes due to its contrast with the white shade especially if someone is reading a lot of product descriptions or spending a lot of the time on the site. The contrast of the site is good and strong enough that anyone with normal vision can read everything easily and choosing these colours feels softer and more softer to look at.

While there are differences between my original visual design and the final prototype the evolution of the project was driven by a focus on usability and brand alignment. In my initial designs I had planned for a brighter green more white space and images on the home page. But once I began building the prototype in React and JavaScript and testing it on actual screens I realized that the brighter green was too harsh, didn't work as I hoped and made the text difficult to read. This is why I chose deepest forest green which is much more better, calming and fits the local brand identity. I also made significant adjustments to the product cards. My original design features had rounded corners and shadow effects but this approach felt disorganised when viewing all the products simultaneously. Instead of these I simplified the cards in the final prototype to ensure the interface is cleaner and easier to scan prioritising functionality over fancy decorations. I kept the overall layout consistent with the navigation bar at the top and the footer at the bottom, I made the product page more straightforward and the buttons more obvious to improve the user experience. I added a dark mode feature during development that I didn't add on my Visual Designs but I did talk about it in Task 1. This was

added based on market research conducted by the client indicating that accessibility features were a high priority for customers. these features and adjustments ensure that the product works better for real people using it on real devices.

The research conducted by the client had multiple features that they wanted in their website, From their research I only added one accessibility feature which was Dark and Light mode and the Loyalty points scheme because of time constraint. For the dark mode I added a toggle that change the website to a dark background with light text. This is helpful for people who find brighter screens painful to look at and also for those who are using the website at night like if they are ordering groceries on bed. The dark mode works on each and every single page. For the loyalty scheme customers can earn 10 points for every £1 they spend. If a customer spend £10.00 on a basket of items they get 100 points added to their account. These points can be used to get discounts on future orders. The reward work like this 100 points gets customers £5 off their next order, 250 points gets them free delivery and 500 points gets them 15 per cent off your whole order. I added a section on the checkout page where customers can see their current points balance and choose which reward they want to use. Once they choose and apply the points the total prices updates straight away so they can see how much they are saving. The points are stored in the database and update automatically everytime someone places an order. I specifically added this loyalty scheme feature as it will encourage more visitors and eventually turning them into GLH customers. This will also bring more customers in, it will build the loyalty and trust as they will keep coming back to the website rather than shopping somewhere else.

I tested the website on different devices by resizing my browser window. On the computer screen the products show up in three or four columns and they change on every device type to meet the user needs and requirements. On phone or tablet the navigation also turns into a hamburger menu with three lines that you click to open different pages. The website design is fully responsive meaning a customer can shop on their phone while they are out or anywhere and no matter what device they are using, the website will work properly.

The products page has a search bar and category buttons right at the top. A customer will not have to scroll through everything to find what they want. They can just type "sourdough" or click "Bakery" and the page immediately shows only the relevant products. This works with each product and each category. This saves time and stops customers from getting annoyed and gives the website a professional look.

The basket page on the site shows a clear list of everything that the customer added. It has plus (+) and (-) buttons to change quantities of the product and a remove button to take things out or remove the products completely and the total updates instantly. The checkout page only asks for an address if the customer picks home delivery. If they pick click and collect it asks for a time slot instead. This means the customers will only have to fill out the fields that are relevant to their orders and picking. This keeps the process very quick and simple.

The producer dashboard is where farmers and food makers manage their products and orders after registering on the website. After the registration, when a producer logs in they only see their own products and not others producers. The dashboard provides the producer with a table with the product name, price, stock level and status. Next to each product, there are input boxes where the producer can change their price or the stock number of the product. These updates save straight away without needing to click a separate button. There is also an "Add New Product" button that opens a simple form to add new items to the shop. The dashboard has two tabs. The first tab shows the producer's products and the second tab shows all customer orders that include their products and the producer can update the order status from "pending" to "preparing" to "ready to collection". This helps customers track their orders and also gives the producers full control over their stock.

The website follows important UX laws that make it easier for customers to use. Fitts's Law says that buttons should be big enough and placed close enough that users can click them without effort. On my website the "Add to Cart" button is large and green and it appears in the same spot on every product card. The "Login" and "Register" buttons are at the top right, which is exactly where most people expect to find them. Hicks law says that if you give people too many choices they take longer to decide. I followed this by adding category filters and a search bar on the products page. Instead of scrolling through all fourteen products, a customer can click "Vegetables" and see only the vegetable products or type "sourdough" into the search bar and see only the breads that match. This narrows down the choices and helps people find what they want faster. Jakobs Law states that users prefer websites to work the same way as other websites they already know. I followed this by putting the logo at the top left, the navigation bar across the top, the basket icon where people expect it and the footer at the bottom with quick links. The website layout is consistent and matches the others professional websites so new customers do not have to learn anything new. The Law of Proximity states that items that are related to each other should be grouped together. On the checkout page all the delivery method options are in one box, all the address fields are in another box and the order summary is in its own box on the right. On the product page the product name, description, price, stock level, and Add to Cart button are all placed close together so the customer sees them as one related group. Millers Law says that people can only hold about seven items in their short term memory at once. I kept this in mind by limiting the number of category filters to six and breaking the checkout process into clear sections and showing basket items in a simple list with quantities and prices. The customer never has to remember too much information at the same time. Parkinson Law says that work expands to fill the time available but in UX terms it means tasks should not take longer than necessary. I made sure the checkout process is quick and simple. A customer can add items to their basket, click checkout, choose delivery or collection and place their order in under a minute. The loyalty points are calculated automatically and the stock updates automatically so the customer does not have to wait or do anything extra.

Legal, Ethical and Regulatory Compliance:

I made sure that my prototype followed the law and good ethical practice throughout the development process. There are several important rules and standards that were considered including the Data Protection Act 2018 and UK GDPR, the Equality Act 2010, copyright laws, accessibility standards and security measures to keep user information safe.

Data Protection Act 2018 and UK GDPR:

I followed these rules in several ways so I only collected the data that I actually needed. My website asks for their name, email address and password when a user registers. No other unnecessary details like the user's date of birth or home address have been asked unless they choose delivery option. This follows the minimisation principle which means you only collect what you need. Secondly, all the passwords are hashed using bcrypt before they are saved into the database meaning even if someone hacked into the database they will only see a long string of random characters and not the actual password. Users have control over their data as they can log in and see their order history and their loyalty points balance. In the future developments, adding a way for users to request that their account be deleted will be added as it is required under the right to erasure in UK GDPR.

Equality Act 2010:

The Equality Act 2010 says that service providers must not discriminate against people with disabilities. For websites, it means the website should be accessible to all users especially those with visual impairments, mobility issues or any other disabilities. I followed the Equality Act by adding an accessibility feature. The dark mode toggle helps people who find bright screens painful to look at and all the images have alt text so a screen reader can describe them to users who can't see the screen. The colour contrast between the text and the background is strong enough that people with vision problems can still read everything. These adjustments mean that disabled users are equally treated and the website is usable by as many as possible.

Copyright and Asset Licensing:

I made sure that all the images and the assets I used were allowed for my project. All product images came from Pexels.com which is a website that offers free photos for commercial and educational use. I did not need to pay for any license or give credit. The map I used on the About Page comes from Leaflet and Open Street map which are open source and free to use. If I had used something that belonged to someone else I could have broken copyright law which is a civil offence.

Accessibility Standard (WCAG 2.1):

The Web Content Accessibility Guidelines, WCAG, are international standards that make websites usable for people with disabilities. Following these guidelines also helps me meet the requirements of the Equality Act 2010. My website meets several WCAG 2.1 guidelines. The colour contrast is strong enough for people with vision problems. All product images have alt

text so screen readers can describe themDark mode is not required by WCAG, but I added it as an extra feature to make the website more comfortable for a wider range of users.

Data Security:

Keeping user data secure is a legal requirement under the Data Protection Act 2018. If a data breach happened and customer information was stolen GLH could face heavy fines from the Information Commissioners Office or ICO. I used several security measures to prevent this. Passwords are hashed using bcrypt, so they cannot be read even if someone stole the database. I used parameterised SQL queries which stops SQL injection attacks where someone tries to hack the database by typing malicious code into a form. I also added validation on both the frontend and the backend. The frontend checks that fields are filled in correctly before sending anything to the server and the backend checks again in case someone tries to bypass the frontend. This double-checking makes the website much more secure.